

Week 2 • Day 3

Logistic Regression — Baseline Model

Trained the baseline Logistic Regression model and mastered classification evaluation metrics

DATE	INTERN NAME	PROJECT	MENTOR
Day 10 of 28 Week 2		Customer Churn Prediction & LTV Engine	

■ Technologies Used Today

Python

Scikit-Learn

Pandas

Matplotlib

Seaborn

joblib

■ Today's Key Metrics

1

Model Trained

>0.75

AUC-ROC Achieved

1,000

Max Iterations Set

2

Evaluation Plots

■ Day Objectives

- Understand Logistic Regression for binary classification
- Train LR with `class_weight='balanced'` for imbalanced data
- Calculate accuracy, precision, recall, F1-score, AUC-ROC
- Plot confusion matrix and ROC curve
- Save the trained model and understand metric tradeoffs

■ Tasks Completed Today

1. Created `src/models/logistic_regression.py`
2. Loaded preprocessed data and called `preprocess_for_ml()`

3. Trained LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
4. Generated predictions: y_pred (class labels) and y_prob (churn probabilities)
5. Calculated full classification report: precision, recall, F1 for both classes
6. Calculated AUC-ROC score using roc_auc_score(y_test, y_prob)
7. Plotted confusion matrix: TP, TN, FP, FN counts visualised
8. Plotted ROC curve: False Positive Rate vs True Positive Rate
9. Saved reports/figures/10_lr_evaluation.png — confusion matrix + ROC
10. Saved models/logistic_regression.pkl using joblib
11. Documented metric results in code comments
12. Committed to GitHub

■ Concepts Learned

- Logistic Regression: outputs probability via sigmoid function, threshold at 0.5
- Confusion matrix: TP=correctly predicted churn, FP=false alarm, FN=missed churner, TN=correctly predicted no churn
- Precision = $TP/(TP+FP)$: of all we flagged as churners, how many actually churned?
- Recall = $TP/(TP+FN)$: of all actual churners, how many did we catch?
- F1 = harmonic mean of precision and recall — the balanced metric
- AUC-ROC: model with 0.5 AUC is no better than random guessing
- class_weight='balanced': adjusts loss function to penalise missing the minority class
- Why recall is more business-critical than precision for churn (missing a churner is costly)

■ Tools / Software Used

Tool / Library	Version / Notes	Status
Scikit-Learn LogisticRegression	1.3.2	✓ Trained
Scikit-Learn metrics	1.3.2	✓ roc_auc_score, classification_report
joblib	1.3.2	✓ Model saved

■ Outputs / Deliverables Produced

- ◆ src/models/logistic_regression.py — training script
- ◆ models/logistic_regression.pkl — saved trained model
- ◆ reports/figures/10_lr_evaluation.png — confusion matrix + ROC curve
- ◆ Classification report printed to terminal
- ◆ AUC-ROC score documented
- ◆ GitHub commit with model and chart

■ Problems Encountered & Solutions

Problem Faced	How I Solved It
ConvergenceWarning: model did not converge	Increased max_iter from default 100 to 1000 — model converged successfully

■ GitHub Commit Record

Commit Message	Week2-Day3: Logistic Regression baseline — trained, evaluated, confusion matrix and ROC saved
Branch	main
Files Changed	src/models/logistic_regression.py, models/logistic_regression.pkl, reports/figures/10_lr_evaluation.png
Commit Hash	(fill after push)

■ End-of-Day Submission Checklist

■ LR model trained successfully	■ AUC-ROC > 0.75
■ Confusion matrix plotted	■ ROC curve plotted
■ Classification report printed	■ Precision/Recall documented
■ logistic_regression.pkl saved	■ Chart 10 saved
■ No convergence warning	■ Day 10 commit on GitHub

■ Self Assessment

8/10 Self Rating	6 Hours Spent	Week 2, Day 3 Progress	10/28 Day in Program
---------------------	------------------	---------------------------------	-------------------------

■ Key Learnings & Reflections

- ★ AUC-ROC is the right metric for imbalanced classification — accuracy is misleading
- ★ A model predicting 'No Churn' always gets 73.5% accuracy — meaningless for this task
- ★ Logistic Regression is a strong interpretable baseline — not always the best but always the starting point

★ max_iter=1000 is usually safe for LR — always check for convergence warnings

■ Plan for Tomorrow

- Train Random Forest Classifier with 200 trees
- Extract and visualise feature importance scores
- Compare RF performance with Logistic Regression
- Save models/random_forest.pkl

INTERN SIGNATURE	MENTOR SIGNATURE	DATE SUBMITTED